

PROJECT REPORT

Phishing Link Detector

A Chrome extension that identifies phishing links on hover, before they are clicked, using local heuristic analysis of the link address.

AUTHOR	DATE	VERSION	LICENCE
FuadAhamed91	15 September 2026	1.1.0	MIT

REPOSITORY

<https://github.com/FuadAhamed91/phishing-detector-extension>

1 Executive summary

Phishing Link Detector is a browser extension for Google Chrome, built on Manifest V3, that evaluates a hyperlink while the cursor rests on it and reports — before any click — where the link actually leads, a safety score from 1 to 100, and the reasons for that score. The analysis is performed entirely within the browser: no addresses, page content or usage data are transmitted to any external service.

The detection engine examines the structure of a URL rather than its reputation. It isolates the domain that was actually registered, checks for well-known brand names used on domains those brands do not own, recognises look-alike spellings (including digits substituted for letters and non-Latin characters that are visually identical to Latin ones), unwraps the redirect links that mail clients place around every hyperlink, and applies a further set of rules covering raw IP addresses, embedded credentials, abused top-level domains and suspicious vocabulary. Results are graded into four tiers and presented on a card beside the cursor, together with the destination domain and up to three reasons.

The engine is implemented as dependency-free JavaScript modules and is covered by 72 automated tests that run in continuous integration on every commit. In a side-by-side comparison with the initial six-rule prototype, the current engine correctly clears legitimate sign-in pages that the prototype had flagged, and identifies four common phishing constructions — mail-client wrappers, the `user@host` trick, digit substitution and Cyrillic homographs — that the prototype had rated as completely safe.

2 Background

Phishing depends on a gap in the browsing experience: the text of a hyperlink and its destination are unrelated, and the destination is not visible until the link has been followed. Browsers show the target address in the status bar on hover, but only as a raw string, and three widely used constructions defeat a quick reading of it.

- **Brand names placed in the subdomain.** The address `paypal.com.account-services.net` begins with a familiar name, but the registered domain is `account-services.net`. Ordinary readers stop at the first familiar words.
- **Look-alike spellings.** `amaz0n.com`, `rnicrosoft.com` and internationalised domains such as `xn--80ak6aa92e.com` — rendered by the browser as `appIe.com` in Cyrillic — are indistinguishable from the genuine names at reading speed.
- **Mail-client wrappers.** Gmail, Google Docs and Microsoft Outlook route outbound links through their own redirect services (`google.com/url?q=...`, Safe Links), so the visible address begins with a trusted domain regardless of the final destination.

A useful tool therefore has to do more than display the address: it has to read it the way an analyst would, and explain its conclusion in plain language at the moment the user is deciding whether to click.

3 Objectives

The project was defined by the following requirements.

1. **Hover-time evaluation.** The verdict must appear while the cursor rests on a link, after a short delay that prevents work on incidental mouse movement, and must never interfere with clicking.
2. **Local processing.** No network requests, accounts or telemetry. Every decision is derived from the link address alone.
3. **Explainability.** Each point deducted corresponds to a reason the user can read, and the card names the domain the link truly leads to.
4. **Precision.** Legitimate sign-in pages such as `accounts.google.com` must not be flagged; every rule is weighed against the false positives it could produce.
5. **Verifiability.** The detection logic must be testable outside the browser and exercised automatically on every change.

4 Solution overview

When the cursor rests on a link for 400 milliseconds, the extension resolves the link's address, submits it to a background service worker for analysis and displays the result on a small card positioned beside the cursor. The card shows the verdict and score, the registered domain of the destination, and up to three reasons; it follows the cursor, keeps clear of the screen edges, and is removed as soon as the cursor leaves the link. Four states are used: *Looks safe* (score 75–100), *Use caution* (50–74), *Suspicious link* (1–49) and *Shortened link*, for URL shorteners whose destination cannot be determined and for which no score is given. Figures 1 and 2 show the extension in use.

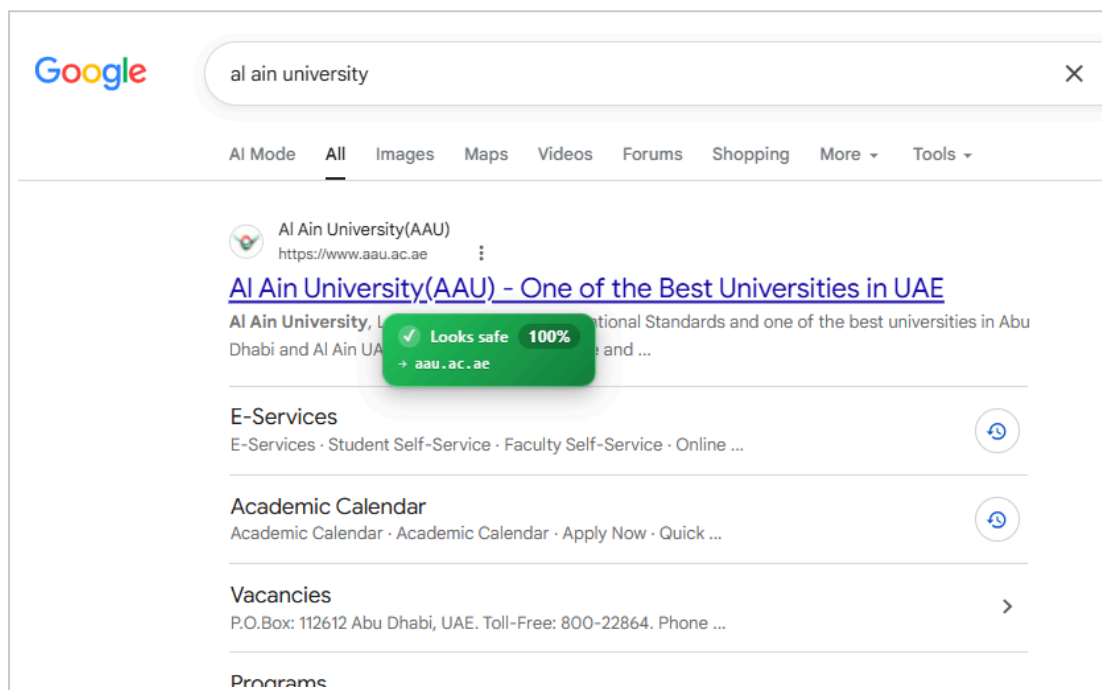


Figure 1. A Google search result for Al Ain University. The card confirms that the link leads to `aau.ac.ae` over HTTPS and that no rule applies: *Looks safe*, 100%.

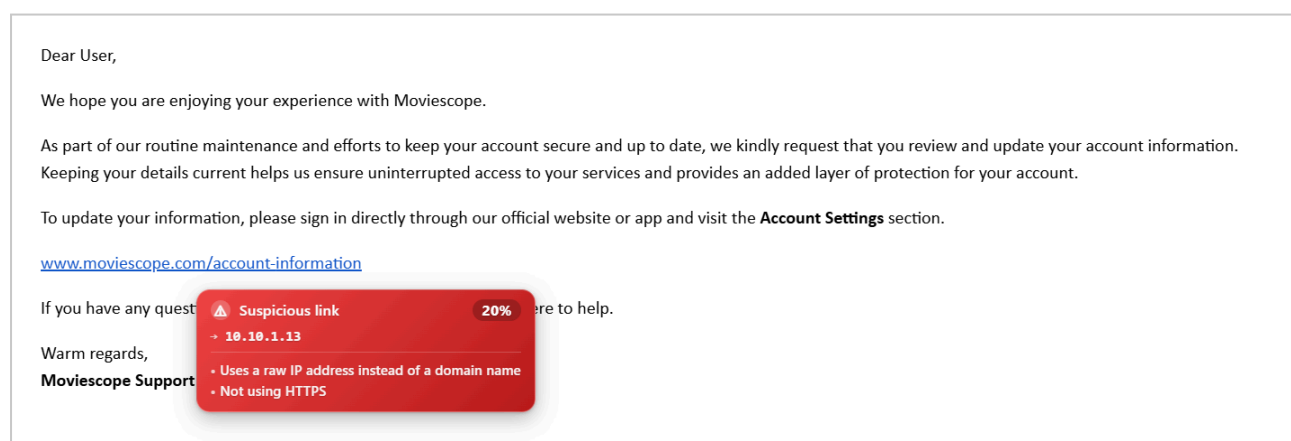


Figure 2. An "update your account information" email. The visible link text is `www.moviescope.com/account-information`; the card shows that the link actually leads to the raw IP address `10.10.1.13` over plain HTTP: *Suspicious link*, 20%. The verdict is available without clicking.

5 Detection method

5.1 Identifying the registered domain

All rules operate on the part of a hostname that its registrant controls. The hostname is matched against a public-suffix list — the catalogue of endings under which the public can register names, such as `com`, `co.uk` or `github.io` — and the label immediately before the suffix is taken as the registered domain. Everything to its left is a subdomain chosen by the same registrant. The engine embeds a curated subset of 414 suffixes covering country-code second levels, hosting platforms that assign subdomains to customers, and dynamic-DNS providers. Table 1 shows the effect.

HOSTNAME	PUBLIC SUFFIX	REGISTERED DOMAIN	SUBDOMAIN
accounts.google.com	com	google.com	accounts
shop.example.co.uk	co.uk	example.co.uk	shop
paypal.com.account-services.net	net	account-services.net	paypal.com
paypal-verify.blogspot.com	blogspot.com	paypal-verify.blogspot.com	—
bucket.s3.amazonaws.com	s3.amazonaws.com	bucket.s3.amazonaws.com	—

Table 1. Hostname decomposition. A brand name in the subdomain (row 3) carries no authority; a customer label on a hosting platform (rows 4–5) is treated as the registered domain.

5.2 Redirect wrappers and URL shorteners

Before any rule runs, the engine recognises ten redirect wrappers — Google's `/url` and AMP endpoints, YouTube, Microsoft Safe Links, Facebook and Instagram, Slack, Reddit, Tumblr, Barracuda and Proofpoint (versions 2 and 3) — and extracts the destination they carry, peeling up to three nested layers. The destination is scored and each wrapper is listed on the card. Forty-two URL shorteners conceal their destination altogether; for these the card reports *Shortened link* with no score rather than a misleading number.

5.3 Brand impersonation and look-alike detection

The engine carries a list of 84 frequently impersonated brands across payments, banking, cloud services, parcel delivery, tax authorities and social media. Each brand is associated with the domains its owner legitimately uses, so that `itunes.apple.com`, `outlook.office.com` and `paypalobjects.com` are recognised as genuine. Brand names appearing in subdomains of seventeen large community platforms (for example `apple.stackexchange.com`) are treated as communities rather than impersonation. Four methods are applied to the labels the registrant controls:

- **Direct use.** The brand name appears as a label, within a label (for names of six or more letters), or split across hyphens: `wells-fargo-login` is read as `wellsfargologin`. Shorter names match only as whole labels, so `pineapple.com` is not treated as Apple.
- **Digit and letter-pair substitution.** The digits 0, 1, 3, 4, 5, 7, 8 and 9 are read as the letters they resemble, and the pairs `rn` and `vv` as `m` and `w`: `amaz0n`, `paypa1`, `rnicrosoft`.
- **Single typing error.** A label at an edit distance of exactly one from a brand name of six or more letters — one inserted, omitted, substituted or transposed character — such as `googel`, `linkedln` or `facebok`. A stop-list excludes ordinary words that happen to sit one letter from a brand: `finance` (Binance), `ccloud` (iCloud), `papa1` (PayPal).
- **Non-Latin homographs.** Punycode labels are decoded (RFC 3492), diacritics are removed, and Cyrillic, Greek, Armenian and Latin-variant characters that are visually identical to Latin letters are mapped to them. `xn--80ak6aa92e` decodes to `appIe` and normalises to `apple`. The card reports the punycode form, since the decoded form would look exactly like the brand.

5.4 Scoring rules

Every address starts at 100 and loses points for each rule that applies. The rules are listed in Table 2 in descending order of weight, which is also the order in which reasons appear on the card. The result is limited to the range 1–100.

RULE	CONDITION	EXAMPLE	PENALTY
Look-alike domain	A label imitates a known brand through a single typing error, substituted digits or letter pairs, or non-Latin characters	amaz0n.com, googel.com, xn-80ak6aa92e.com	60
Fake host in user information	A domain name appears before @; the real host follows it	https://google.com@evil.com/	60
IP address host	The host is a raw IPv4 or IPv6 literal	http://10.10.1.13/login	60
Brand impersonation	A known brand name on a domain the brand does not own	paypal.com.evil.net, secure-paypal.com	50
Abused top-level domain	One of 44 TLDs that dominate published abuse statistics	.xyz, .top, .tk, .click	40
Keyword in registered domain	login, signin, secure, verify, account, update, password or webmail	secure-login.com	30
Embedded credentials	A username and password in the address	https://admin:pw@example.com/	30
Insecure protocol	http:// rather than https://	http://example.com/	20
Excessive hyphens	More than two hyphens in the hostname	my-bank-secure-login.xyz	20
Long hostname	More than 40 characters	—	15
International characters	Any punycode (xn--) label	xn--mller-kva.de	15
Keyword in subdomain	A listed keyword only in a subdomain, where legitimate services place it	accounts.google.com	10

Table 2. The twelve scoring rules. Any rule weighted 60 is enough on its own for *Suspicious link*; brand impersonation or an abused TLD alone produces *Use caution*; the lighter rules only change the verdict in combination.

6 Architecture and implementation

The extension comprises two isolated contexts that communicate through a single message. A content script runs inside every page, observes hover events and draws the card; a background service worker owns the detection engine. The content script never sees the rule set and the service worker never touches the page.

STEP 1 Observe The content script listens for hover events on the document body and ignores empty, in-page and javascript: links. Relative addresses are resolved against the page.	STEP 2 Debounce After 400 ms without leaving the link, the resolved address is sent to the service worker. Replies for links the cursor has already left are discarded.	STEP 3 Analyse The service worker unwraps redirectors, decomposes the hostname, applies the twelve rules and returns the verdict, score, destination, reasons and wrapper hops.	STEP 4 Present The card is drawn at the cursor with text-only insertion, positioned within the viewport, and removed when the cursor leaves the link.
--	--	--	--

FILE	LINES	RESPONSIBILITY
manifest.json	30	Manifest V3 declaration; registers the module service worker and the content script and stylesheet on all URLs
content.js	216	Event delegation, link filtering, debounce, card rendering and positioning
styles.css	180	Card styling; fixed positioning, maximum stacking order and pointer transparency are enforced so page styles cannot override them
background.js	15	Message listener; the only file that references the extension API
scoring.js	231	Hostname decomposition, the rule table, tiers and analyzeUrl()
lookalike.js	213	Punycode decoding, character normalisation, substitution and edit-distance matching
redirects.js	93	Redirect wrapper patterns and unwrapping
lists.js	222	Data: public suffixes, brands, keywords, abused TLDs, shorteners, stop-lists
test/	474	Automated tests (five files)

Table 3. Source files at version 1.1.0: approximately 1,200 lines of extension code and 470 lines of tests, with no third-party dependencies.

7 Testing and verification

Because the engine has no dependency on browser APIs, it is tested directly in Node.js with the built-in test runner. The suite of 72 tests (Table 4) runs in under a second locally and in GitHub Actions on every push to the main branch; all runs to date have passed.

TEST FILE	TESTS	SCOPE
scoring.test.js	44	IP detection, hostname decomposition, each rule in isolation, tiers, clamping, invalid input, redirect and shortener verdicts, and fourteen realistic addresses pinned to their tier
lookalike.test.js	14	Punycode decoding, character normalisation, and each brand-detection method with its false-positive guards
redirects.test.js	8	Every wrapper pattern, nested wrappers, the hop limit, Proofpoint decoding, rejection of non-HTTP destinations
background.test.js	3	The message contract between content script and service worker
manifest.test.js	3	Manifest validity and the presence of every referenced file

Table 4. Automated test coverage by file.

Two further checks were made. The punycode decoder was validated against Node.js's own internationalised-domain implementation for eight labels in Latin, Cyrillic, Greek, Chinese and Japanese scripts. The card was exercised in a browser through a harness page running the production content script and stylesheet against the engine: all four states rendered as designed, the card repositioned correctly at the viewport edges, and a deliberately hostile page style rule left it unchanged.

8 Results

Table 5 compares the initial six-rule prototype (version 1.0) with the current engine on twelve addresses: four legitimate, eight phishing constructions.

ADDRESS	V1.0	V1.1	REPORTED DESTINATION
https://github.com/	100 safe	100 safe	github.com
https://accounts.google.com/	70 warning	90 safe	google.com
https://login.microsoftonline.com/	70 warning	90 safe	microsoftonline.com
http://example.com/	80 safe	80 safe	example.com
https://www.google.com/url?q=https://evil.xyz	100 safe	60 caution	evil.xyz
https://paypal.com.account-services.net/	70 warning	20 warning	account-services.net
https://paypal.com-secure.net/	70 warning	20 warning	com-secure.net
https://google.com@evil.com/	100 safe	40 warning	evil.com
https://amaz0n.com/	100 safe	40 warning	amaz0n.com
https://xn--80ak6aa92e.com/	100 safe	25 warning	xn--80ak6aa92e.com
https://wells-fargo-login.com/	70 warning	20 warning	wells-fargo-login.com
https://bit.ly/3xyz	100 safe	unknown	bit.ly

Table 5. Verdicts from both engine versions, produced by running the committed code of each.

9 Conclusion and future work

The project set out to give users a trustworthy answer to one question — *where does this link really go, and does it look like a phishing link?* — at the moment they are deciding whether to click, without sending their browsing anywhere. The delivered extension answers it in a form that is explainable (every deduction is a sentence on the card), precise (legitimate sign-in pages are no longer flagged) and verifiable (the engine is pure code under 72 automated tests). The move from whole-hostname string matching to analysis of the registered domain, together with brand and homograph detection and redirect unwrapping, is what turned an indicative prototype into a tool that can be relied on.

Planned developments, in order of value:

1. **Link text against destination.** Sending the visible text of the link to the engine allows the classic email construction — text that reads as one address while the link leads to another — to be scored directly.
2. **Keyboard support.** Presenting the card on focus as well as hover, so that users who navigate with the keyboard receive the same warning.
3. **An options page** with a personal allow-list and adjustable thresholds, together with icons and an action popup in preparation for a Chrome Web Store listing.
4. **Optional reputation lookup** against a published phishing feed as an opt-in feature, complementing structural analysis while preserving the local-only default.

References

- [1] Phishing Link Detector source repository. <https://github.com/FuadAhamed91/phishing-detector-extension>
- [2] Google. *Chrome Extensions: Manifest V3*. <https://developer.chrome.com/docs/extensions>
- [3] Mozilla Foundation. *Public Suffix List*. <https://publicsuffix.org>
- [4] A. Costello. *RFC 3492: Punycode — A Bootstring encoding of Unicode for Internationalized Domain Names in Applications*. IETF, 2003.
- [5] The Unicode Consortium. *Unicode Technical Standard #39: Unicode Security Mechanisms*. <https://unicode.org/reports/tr39>
- [6] Spamhaus. *The World's Most Abused TLDs*. <https://www.spamhaus.org/statistics/tlds>

A Appendix: installation

1. Clone or download the repository from the address on the title page.
2. Open `chrome://extensions` in Google Chrome and enable **Developer mode**.
3. Choose **Load unpacked** and select the folder containing `manifest.json`.
4. Rest the cursor on any link. After a file is edited, reload the extension from the same page and refresh open tabs.

The automated tests are run with `npm test` and require Node.js 20 or later.